

Introduction to Data Analysis

Week 2. Data Types and Data Structures

Jin Hee Yoon

2026

Sejong University

Week 2 Overview

1주차에서 데이터분석의 전체 흐름을 살펴보았다면, 2주차에서는 데이터가 어떤 유형과 구조로 저장되어 있는가를 다룬다.

- **Week 2-1:** 여덟 가지 주제를 개념으로 설명하고, 예제 표에서 문제를 직접 찾아본다.
- **Week 2-2:** 같은 여덟 가지 주제를 같은 순서로 Python 코드로 실습한다.

Topics of Week 2

	주제	Week 2-1 (개념)	Week 2-2 (실습)
1	변수의 척도	네 가지 측정 척도	dtype 확인, category 지정
2	저장 타입	저장 형식과 의미의 불일치	수치 · 날짜 타입 변환
3	결측값	결측의 표현과 처리 원칙	결측 확인, 대체와 표시
4	행의 식별	index와 key	loc, iloc, 중복 확인
5	Structured data	정리된 표의 세 원칙	한 열에 섞인 변수 분리
6	Wide와 Long	같은 정보, 다른 배치	melt, pivot
7	표의 결합	key와 join의 종류	merge, 행 수 확인
8	그룹 비교	요약과 되돌리기	groupby, agg, transform

Week 2-1. 데이터의 유형과 구조

The Example Table

다음은 어느 강의의 수강생 학습 기록을 정리한 표의 예이다. 한 행은 학생 한 명을 나타낸다.

ID	MajorGrade	PriorLevel	StudyTime	JoinDate	Score
201	AI-2	low	120	2026-03-02	78
202	AI-2	high	150	2026-03-02	85
203	Data-3	medium	unknown	2026-03-09	92
204	Data-3	high	-99	2026-03-09	70
205	Design-1	low	90	2026-03-16	88
203	Data-3	medium	unknown	2026-03-09	92

값을 읽기 전에 각 변수가 무엇을 의미하는지 먼저 확인해야 한다. 이 데이터에는 여러 가지 생각해 볼 사항들을 가지고 있다.

Variables in the Example Table

Variable	의미	단위 및 형식	척도
ID	학생 식별번호	정수	Identifier
MajorGrade	전공과 학년	“전공-학년” 형태의 문자	두 변수가 결합됨
PriorLevel	수강 시작 시점의 사전 수준	low, medium, high	Ordinal
StudyTime	주당 학습 시간	분 단위	Ratio
JoinDate	수강 신청일	YYYY-MM-DD	Interval
Score	기말 점수	점, 100점 만점	Interval

- ID는 숫자로 저장되지만 크기에 의미가 없으므로 평균을 계산하지 않는다.
- StudyTime의 120은 분 단위이므로 2시간을 뜻한다. 단위를 모르면 해석이 달라진다.
- PriorLevel은 순서가 있지만 단계 사이의 간격이 같다고 보기는 어렵다.

이 장의 설명이 곧 **data dictionary**이다. 이 정보가 없으면 같은 표를 보고도 서로 다르게 해석하게 된다.

Topic 1. Scales of Measurement

변수의 유형은 “숫자인가 문자인가”가 아니라 어떻게 측정되었는가로 판단한다.

Scale	의미	예
Nominal	구분만 가능, 순서 없음	전공, 지역, 혈액형
Ordinal	순서는 있으나 간격은 불분명	만족도, 학년, 수준
Interval	차이는 의미 있으나 0이 "없음"이 아님	온도, 년도, 시험 점수
Ratio	0이 "없음"이며 배수 비교 가능	학습 시간, 금액, 거리

0이 진짜 "없음"을 뜻하면 ratio, 0이 그냥 임의로 정한 눈금 위치면 interval

예제 표에서 PriorLevel은 ordinal, MajorGradesms nominal, StudyTime은 ratio, Score는 interval로 볼 수 있다.

Topic 1. Why the Scale Matters

척도가 정해지면 사용할 수 있는 요약 방법이 결정된다.

Scale	적절한 요약	부적절한 요약
Nominal	빈도, 비율, 최빈값	평균
Ordinal	중앙값, 빈도	표준편차
Interval	평균, 표준편차	배수 비교
Ratio	평균, 배수 비교	-

Topic 2. Storage Type and Meaning

컴퓨터가 값을 저장한 형식과, 그 값이 실제로 의미하는 바는 다를 수 있다.

열	저장된 형식	실제 의미	문제가 되는 상황
ID	숫자	식별번호	평균을 계산할 때
StudyTime	문자	시간(수치)	합계를 구할 때
JoinDate	문자	날짜	순서대로 정렬할 때
PriorLevel	문자	순서가 있는 범주	크기를 숫자처럼 비교할 때

분석 오류의 상당수는 계산 방법이 아니라, 저장 형식을 실제 의미로 착각한 데서 생긴다. 예제 표의 StudyTime은 숫자처럼 보이지만 “unknown”이 섞여 있어 문자로 저장된다. low, medium, high를 1, 2, 3으로 바꾸는 순간 간격까지 같다고 가정하게 되는 문제가 생긴다. 순서는 쓰되, 간격은 쓰지 않는다.

Topic 2. Dates Are Not Text

날짜를 "문자"로 저장하면 다음과 같은 일이 생긴다.

- “2026-03-09”와 “2026-03-02”의 차이가 며칠인지 계산할 수 없다. (문자의 나열)
- “3/9/2026”처럼 형식이 다르면 혼동이 있다. (미국식 or 유럽식,한국식)
- 연도나 요일을 따로 꺼내 쓸 수 없다.

"날짜 타입"으로 바꾸어 두면 날짜 간 차이 계산, 연도 추출, 기간 비교가 모두 가능해진다.

데이터를 불러온 직후에 열별 저장 형식을 확인하는 것은, 요약 통계량을 계산하기 전에 반드시 거쳐야 하는 단계이다. Pandas 는 default 가 미국식(M/D/Y)임, 유럽,한국식(D/M/Y) 사용 시 `dayfirst=True` 를 추가해야 한다.

Topic 3. How Missing Values Appear

결측값은 하나의 모습으로 나타나지 않는다.

표현	예	주의점
빈 칸	(공백)	읽는 방식에 따라 다르게 처리됨
문자열	unknown, 미상, -	숫자 열이 문자 열이 됨
약속된 숫자	-99, 9999	그대로 계산하면 평균이 왜곡됨
0	0	실제 0과 구분되지 않음

예제 표의 StudyTime에는 “unknown”과 -99가 함께 들어 있다. 두 값 모두 실제 학습 시간이 아니라 “측정되지 않음”을 뜻한다.

Topic 3. Handling Missing Values

결측을 어떻게 다룰지에는 정답이 없고, 선택과 근거가 있을 뿐이다.

방법	장점	주의점
그 행을 제외	단순함	표본이 줄고 결과가 치우칠 수 있음
대표값으로 대체	표본 유지	값의 흩어진 정도가 실제보다 작아짐
그룹별 대표값	집단 차이 반영	그룹이 작으면 불안정함
대체 여부를 표시	정보 보존	열이 하나 늘어남

결측을 0으로 채우면 “측정하지 않음”과 “측정 결과가 0”을 영원히 구별할 수 없게 된다. 어떤 방법을 쓰든 결과 보고에 처리 방식을 밝혀야 한다.

Topic 4. Two Ways to Point at a Row

표에서 특정 행을 가리키는 방법은 두 가지이다.

Index	ID	MajorGrade	Score
0	201	AI-2	78
1	202	AI-2	85
2	203	Data-3	92

- **Index:** 프로그램이 행을 찾기 위해 붙여 둔 이름표이다.
- **ID:** 데이터 안에 저장된 식별번호로, 사람이 정한 값이다.
- 정렬을 하면 첫 번째 행의 index는 더 이상 0이 아닐 수 있다.

“위치로 세 번째 행”과 “이름표가 3인 행”은 서로 다른 대상일 수 있다. 이 구분이 2-2 실습의 출발점이 된다.

Topic 4. Keys and Duplicates

Key는 한 행을 유일하게 식별하는 열 또는 열의 조합이다.

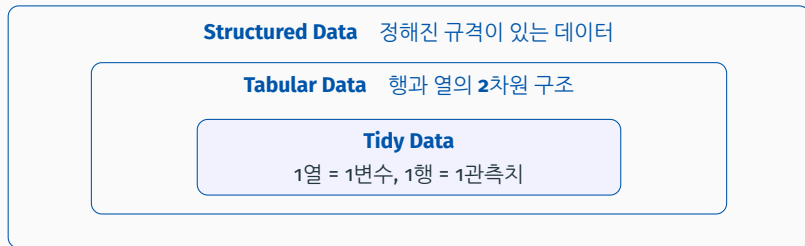
데이터	Key	형태
학생 명단	ID	단일 key
주차별 학습 기록	(ID, Week)	두 열의 조합
수강 신청 내역	(ID, CourseID)	두 열의 조합

예제 표에서 ID 203인 행은 두 번 나타난다. 이 상태로 평균을 계산하면 한 학생이 두 번 반영된다.

Key는 그중에서 행을 유일하게 구분하는 역할을 맡은 열이다. 별도의 열이 아니라, 이미 있는 variable 중 하나(또는 몇 개의 조합)에 붙는 이름표같은 역할을 한다.

Topic 5. Structured, Tabular, and Tidy Data

정형 데이터라고 해서 모두 표는 아니고, 표라고 해서 모두 분석하기 좋은 것은 아니다. 세 개념은 다음과 같은 포함 관계를 이룬다.



개념	조건	예
Structured	미리 정해진 규격을 따름	관계형 DB, 그래프 DB, 표
Tabular	행과 열로 이루어진 2차원 표	CSV, Excel 시트, DataFrame
Tidy	열 하나에 변수 하나, 행 하나에 관측 하나	정리를 마친 분석용 표

Topic 5. Tidy Data

Tidy data는 분석 도구가 바로 다룰 수 있도록 정리된 표를 뜻하며, 세 가지 원칙을 따른다.

1. 하나의 변수는 하나의 열을 차지한다.
2. 하나의 관측은 하나의 행을 차지한다.
3. 관측 단위가 다르면 표를 나눈다.

예제 표의 MajorGrade 열에는 “AI-2”처럼 전공과 학년이라는 두 변수가 함께 들어 있다. 이 상태로는 전공별 비교도, 학년별 비교도 바로 할 수 없다.

Tidy data는 보기 좋은 표와 다르다. 사람이 읽기 편한 표가 분석에는 불편한 경우가 오히려 많다.

Topic 5. Common Violations

유형	예	정리 방법
한 열에 두 변수	AI-2	두 열로 분리
한 칸에 여러 값	“수학, 영어”	여러 행으로 분리
열 이름에 value	Week1, Week2, Week3	Long format으로 변환
관측 단위 혼재	학생 정보와 주차 기록이 한 표에	표를 나눔

이런 표는 잘못 만들어진 것이 아니라, 보고용으로 만들어진 것이다. 분석을 하려면 형식을 바꾸는 과정이 필요할 뿐이다.

Topic 6. Wide and Long

같은 정보라도 배치하는 방법은 두 가지이다.

Wide format			
ID	Week1	Week2	Week3
201	70	75	80
202	65	72	78

Long format		
ID	Week	Score
201	Week1	70
201	Week2	75
201	Week3	80

- Wide는 사람이 한눈에 비교하기 좋고, 보고서 표에 적합하다.
- Long은 그룹별 요약과 그래프 작성에 적합하다.
- Wide에서 한 행은 학생, Long에서 한 행은 학생-주차 조합이다.

Topic 6. Which Format When?

그래프의 결과 자체는 Wide format과 Long format이 동일하다. 작은 데이터에서는 Wide format이 오히려 더 직관적일 수 있다.

Wide format

- 한 행이 한 학생을 나타낸다.
- Week1, Week2, Week3 열의 값을 연결한다.
- 소수 학생의 점수를 직접 비교하기 편하다.

Long format

- 한 행은 한 학생과 특정 주차의 조합이다.
- 변수의 역할을 다음과 같이 지정한다.
 - `x = Week`
 - `y = Score`
 - `group = ID`

같은 데이터를 다르게 배치한 것이므로
두 형식으로 그린 그래프의 결과는 같다.

Topic 6. Which Format When?

- 학생 수가 많아지거나 새로운 주차가 추가되어도 동일한 변수 구조와 그래프 명령을 그대로 사용할 수 있다.

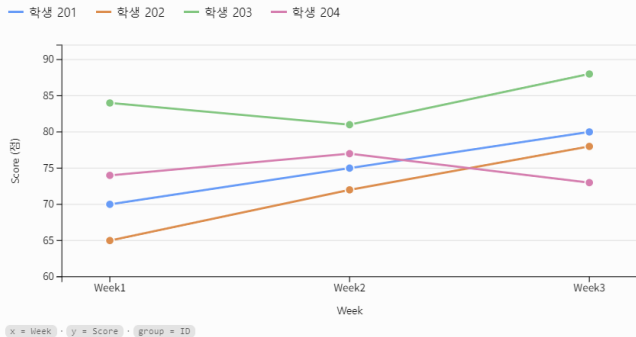
예를 들어 학생이 1,000명이어도 변수의 역할만 지정하면 된다.

`x = Week,      y = Score,      group = ID`

- 학생이 추가되면 새로운 행을 추가한다.
- Week4가 추가되어도 새로운 행을 추가한다.
- Week, Score, ID라는 변수 구조는 변하지 않는다.
- 같은 그래프 명령으로 모든 학생의 변화를 그릴 수 있다.

Long format이 더 좋은 그래프를 만드는 것은 아니다. 많은 관측값을 동일한 규칙으로 처리하기 쉽게 만드는 형식이다.

Topic 6. Which Format When?



- **Wide format:** Week1, Week2, Week3 열을 찾아 각 학생의 한 행씩 연결한다.
- **Long format:** x=Week, y=Score, group=ID와 같이 각 변수의 역할만 지정한다.
- Long format의 장점은 학생 수가 많거나 주차가 추가될 때 나타난다.
- 예를 들어 학생이 1,000명이어도 코드 한 줄 명령으로 모든 학생의 점수 변화를 그릴 수 있다.

Topic 7. Combining Two Tables

현실의 데이터는 하나의 표가 아니라 여러 표로 나뉘어 있는 경우가 많다.

학생 정보		
ID	MajorGrade	PriorLevel
201	AI-2	low
202	AI-2	high

주차별 기록		
ID	Week	Score
201	W1	70
201	W2	75
202	W1	65

- 학생 정보를 주차마다 반복 저장하지 않으므로 수정이 쉽고 오류가 줄어든다.
- 분석할 때 공통 key인 ID를 기준으로 두 표를 이어 붙인다.

Topic 7. Which Rows Survive?

학생 정보와 주차별 기록

두 표는 공통 변수인 ID를 기준으로 결합한다.

학생 정보: 왼쪽 표

ID	MajorGrade	PriorLevel
201	AI-2	low
202	AI-2	high

주차별 기록: 오른쪽 표

ID	Week	Score
201	W1	70
201	W2	75
202	W1	65

- 학생 정보는 학생 한 명당 한 행으로 저장되어 있다.
- 주차별 기록은 학생과 주차의 조합마다 한 행으로 저장되어 있다.
- 결합하면 학생 정보가 각 주차의 기록에 연결된다.

Topic 7. Which Rows Survive?

표를 이어 붙일 때는 어떤 행을 남길지 먼저 정해야 한다.

방식	남는 행	사용하는 경우
Inner	양쪽 모두에 있는 key	두 정보가 모두 필요할 때
Left	왼쪽 표의 모든 key	기준 명단을 유지할 때
Outer	양쪽의 모든 key	누락 자체를 확인할 때

결합한 뒤에는 행 수를 반드시 확인한다. 행이 줄었다면 한쪽에만 있는 key가 있는 것이고, 행이 늘었다면 한쪽 표의 key가 유일하지 않은 것이다.

Topic 7. Which Rows Survive?

두 표를 ID로 결합한 결과

ID	MajorGrade	PriorLevel	Week	Score
201	AI-2	low	W1	70
201	AI-2	low	W2	75
202	AI-2	high	W1	65

- ID 201은 주차별 기록이 두 개이므로 결과에도 두 행이 생성된다.
- ID 202는 주차별 기록이 하나이므로 결과에 한 행이 생성된다.
- 학생 정보가 각 주차별 기록에 자동으로 연결된다.

현재 데이터에서는 모든 **ID**가 양쪽 표에 있으므로
Inner, Left, Outer join의 차이가 나타나지 않는다.

Topic 7. Which Rows Survive?

Join 방식의 차이를 확인하기 위한 예

- 학생 정보에 아직 점수가 없는 ID 203을 추가한다.
- 주차별 기록에 학생 정보가 없는 ID 204를 추가한다.

학생 정보: 왼쪽 표

ID	MajorGrade	PriorLevel
201	AI-2	low
202	AI-2	high
203	AI-1	low

주차별 기록: 오른쪽 표

ID	Week	Score
201	W1	70
201	W2	75
202	W1	65
204	W1	80

- ID 203은 왼쪽 표에만 존재한다.
- ID 204는 오른쪽 표에만 존재한다.

Topic 7. Which Rows Survive?

Inner join: 양쪽 표에 있는 ID만 유지

ID	MajorGrade	PriorLevel	Week	Score
201	AI-2	low	W1	70
201	AI-2	low	W2	75
202	AI-2	high	W1	65

- ID 201과 202는 양쪽 표에 있으므로 남는다.
- ID 203은 주차별 기록이 없으므로 제외된다.
- ID 204는 학생 정보가 없으므로 제외된다.

학생 정보와 주차별 기록이 모두 있는 경우만 분석한다.

Topic 7. Which Rows Survive?

Left join: 학생 정보의 모든 ID 유지

ID	MajorGrade	PriorLevel	Week	Score
201	AI-2	low	W1	70
201	AI-2	low	W2	75
202	AI-2	high	W1	65
203	AI-1	low	NaN	NaN

- 왼쪽 표의 ID 201, 202, 203을 모두 유지한다.
- ID 203은 주차별 기록이 없으므로 NaN이 입력된다.
- 오른쪽 표에만 있는 ID 204는 제외된다.

등록된 학생 명단을 모두 유지할 때 적합하다.

Topic 7. Which Rows Survive?

Outer join: 양쪽 표의 모든 ID 유지

ID	MajorGrade	PriorLevel	Week	Score
201	AI-2	low	W1	70
201	AI-2	low	W2	75
202	AI-2	high	W1	65
203	AI-1	low	NaN	NaN
204	NaN	NaN	W1	80

- 어느 한쪽에라도 존재하는 ID는 모두 남는다.
- ID 203을 통해 점수가 없는 학생을 확인할 수 있다.
- ID 204를 통해 학생 정보가 없는 기록을 확인할 수 있다.

두 표 사이의 누락이나 불일치를 확인할 때 적합하다.

Topic 7. Which Rows Survive?

Join 방식 비교

방식	남는 ID	사용하는 경우
Inner	201, 202	양쪽 정보가 모두 필요할 때
Left	201, 202, 203	학생 명단을 유지할 때
Outer	201, 202, 203, 204	누락과 불일치를 확인할 때

- **Inner:** 양쪽 표에서 일치하는 ID만 남긴다.
- **Left:** 왼쪽 학생 정보의 ID를 모두 남긴다.
- **Outer:** 양쪽 표의 ID를 모두 남긴다.

어떤 행을 반드시 유지해야 하는지 먼저 결정한 후
Join 방식을 선택한다.

Topic 7. Which Rows Survive?

결합 후 행 수가 증가하는 이유

학생 정보에서 ID 201은 한 행이지만, 주차별 기록에는 두 행이 있다.

학생 정보

ID	MajorGrade	PriorLevel
201	AI-2	low

주차별 기록

ID	Week	Score
201	W1	70
201	W2	75

- 결합 결과에서 ID 201은 두 행이 된다.
- 이것은 오류가 아니라 주차별 기록이 두 개이기 때문이다.
- 학생 정보는 각 주차의 행에 반복하여 연결된다.

Topic 7. Which Rows Survive?

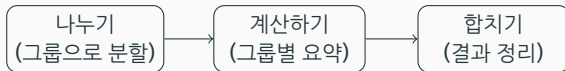
결합 후 반드시 확인할 사항

- 결합 전후의 행 수를 비교한다.
- 행이 줄었다면
 - 한쪽 표에만 있는 ID가 제외되었는지 확인한다.
 - Inner join을 사용했는지 확인한다.
- 행이 늘었다면
 - 동일한 ID가 여러 행에 존재하는지 확인한다.
 - 주차별 반복 기록인지, 잘못된 중복인지 구분한다.
- 결측값이 생겼다면
 - 한쪽 표에 대응되는 ID가 없는지 확인한다.

Join 후에는 행 수, 중복 **key**, 결측값을 확인해야 한다.

Topic 8. Comparing Groups

그룹별로 나누어 보는 작업은 세 단계로 이루어진다.



결과를 받는 방식은 두 가지이다.

- **요약하기**: 그룹당 한 줄로 줄인다. 전공이 세 개면 결과는 세 행이다.
- **되돌리기**: 그룹의 계산 결과를 원래 각 행에 다시 붙인다. 행 수는 그대로이다.

Topic 8. Why "Bringing It Back" Is Useful

전공별 평균 점수가 다음과 같다고 하자.

ID	MajorGrade	Score	전공 평균
201	AI	78	81.5
202	AI	85	81.5
203	Data	92	81.0
204	Data	70	81.0

전공 평균을 각 학생의 행에 되돌려 놓으면, 점수에서 전공 평균을 빼서 **같은 전공 안에서의 상대적 위치**를 계산할 수 있다.

92점은 전체에서도 높은 점수이지만, 소속 전공 안에서 얼마나 높은지는 전공 평균과 비교해야 알 수 있다.

Diagnosing the Example Table

여덟 가지 주제를 예제 표에 적용하면 다음 문제가 드러난다.

Topic	발견한 문제	필요한 조치
1	PriorLevel의 순서 정보가 표시되어 있지 않음	순서를 지정
2	StudyTime과 JoinDate가 문자로 저장됨	타입 변환
3	unknown과 -99가 결측을 뜻함	결측으로 바꾸고 처리
4	ID 203이 중복됨	중복 확인 후 제거
5	MajorGrade에 전공과 학년이 섞여 있음	두 열로 분리
6	주차별 점수는 별도의 wide 표에 있음	Long으로 변환
7	두 표를 이어 붙여야 함	key 기준 결합
8	전공별 비교가 필요함	그룹 요약과 되돌리기

Week 2-1 Summary

- 척도가 그 변수에 쓸 수 있는 요약 방법을 결정한다.
- 저장 형식과 실제 의미는 다를 수 있으므로 반드시 확인한다.
- 결측은 여러 모습으로 나타나며, 처리 방법과 근거를 함께 남긴다.
- 한 행이 무엇인지, 무엇이 key인지가 표 구조의 핵심이다.
- Tidy data 원칙과 wide/long 변환으로 표를 분석 가능한 형태로 만든다.
- 그룹 비교는 요약과 되돌리기 두 방식으로 이루어진다.

2-2에서는 이 여덟 가지를 같은 순서로, 같은 예제 표에 대해 코드로 실행한다.

Week 2-2. Python Practice

Week 2-2 Overview

- 2-1의 Topic 1-8을 실습한다.
- 문제가 있는 예제 표를 만든 뒤, 단계마다 하나씩 정리한다.
- 마지막에는 정리된 데이터로 전공별 비교까지 수행한다.

코드를 외우기보다, 각 단계에서 무엇을 확인하고 무엇이 달라졌는지를 눈으로 확인하는 것이 중요하다. 실행 결과를 항상 출력해 본다.

Setup: The Example Table in Python

```
import numpy as np
import pandas as pd

# 2-1에서 살펴본 예제 표를 그대로 입력한다
df = pd.DataFrame({"ID": [201, 202, 203, 204, 205, 203],
    "MajorGrade": ["AI-2", "AI-2", "Data-3",
        "Data-3", "Design-1", "Data-3"],
    "PriorLevel": ["low", "high", "medium",
        "high", "low", "medium"],
    "StudyTime": ["120", "150", "unknown",
        "-99", "90", "unknown"],
    "JoinDate": ["2026-03-02", "2026-03-02", "2026-03-09",
        "2026-03-09", "2026-03-16", "2026-03-09"],
    "Score": [78, 85, 92, 70, 88, 92] })

print(df)
```

Practice 1. Checking Types and Scales

```
# 열별 저장 형식을 먼저 확인한다
print(df.dtypes)

# 범주형 변수는 평균이 아니라 빈도로 요약한다
print(df["MajorGrade"].value_counts())
print(df["PriorLevel"].value_counts())

# 수치형 변수만 평균을 계산한다
print(df["Score"].mean())
```

StudyTime이 object로 표시되는 것에 주목한다. 숫자처럼 보이지만 문자로 저장되어 있어 평균을 계산할 수 없다.

Practice 1. Recording the Order of Categories

```
# 순서가 있는 범주임을 데이터에 명시한다
df["PriorLevel"] = pd.Categorical(df["PriorLevel"],
                                   categories=["low", "medium", "high"],
                                   ordered=True)

print(df["PriorLevel"])

# 순서를 지정했으므로 크기 비교가 가능해진다
print(df[df["PriorLevel"] >= "medium"])
```

순서를 지정하기 전에는 low, medium, high가 알파벳 순서로 처리된다. 지정한 뒤에는 우리가 정한 순서대로 정렬되고 비교된다.

Practice 2. Converting Numbers and Dates

```
# 숫자로 바꿀 수 없는 값은 오류 대신 결측으로 처리한다
df["StudyTime"] = pd.to_numeric(df["StudyTime"],
                                errors="coerce")

# 문자로 된 날짜를 날짜 타입으로 바꾼다
df["JoinDate"] = pd.to_datetime(df["JoinDate"])

print(df.dtypes)
print(df[["StudyTime", "JoinDate"]])
```

`errors="coerce"`는 “unknown”처럼 바꿀 수 없는 값을 조용히 결측으로 만든다. 편리하지만, 다음 단계에서 결측이 몇 개 생겼는지 반드시 확인해야 한다.

Practice 2. What Dates Make Possible

```
# 날짜에서 필요한 요소를 꺼낸다
df["JoinMonth"] = df["JoinDate"].dt.month
df["JoinWeekday"] = df["JoinDate"].dt.day_name()

# 두 날짜의 차이를 일 단위로 계산한다
gap = pd.Timestamp("2026-06-01") - df["JoinDate"]
df["DaysSinceJoin"] = gap.dt.days

print(df[["ID", "JoinDate", "JoinWeekday", "DaysSinceJoin"]])
```

문자로 저장되어 있을 때는 불가능했던 계산이, 타입을 바꾸는 것만으로 가능해진다.

Practice 3. Finding Missing Values

```
# 약속된 숫자로 표시된 결측을 진짜 결측으로 바꾼다
df["StudyTime"] = df["StudyTime"].replace(-99, np.nan)

# 열별 결측 개수와 비율을 확인한다
print(df.isna().sum())
print(df["StudyTime"].isna().mean())

# 결측값은 등호로 비교할 수 없다
print(np.nan == np.nan)
```

-99를 그대로 두고 평균을 구하면 학습 시간이 실제보다 훨씬 짧게 나온다. 결측을 먼저 정리한 뒤에 계산해야 한다.

Practice 3. Filling with a Reason

```
#대체하기 전에 대체 여부를 표시로 남긴다
df["TimeImputed"] = df["StudyTime"].isna()

#중앙값으로 채운다: 극단값의 영향을 덜 받는다
df["StudyTime"] = df["StudyTime"].fillna(
    df["StudyTime"].median()
)

print(df[["ID", "StudyTime", "TimeImputed"]])
```

표시 열을 남겨 두면, 나중에 대체한 값을 제외한 결과와 비교할 수 있다. 결측을 0으로 채우지 않은 이유는 학습을 전혀 하지 않은 학생과 구분해야 하기 때문이다.

Practice 4. Position and Label

```
sorted_df = df.sort_values("Score", ascending=False)
print(sorted_df[["ID", "Score"]])

# iloc: 위치로 첫 번째 행을 선택한다
print(sorted_df.iloc[0])

# loc: 이름표가 0인 행을 선택한다
print(sorted_df.loc[0])
```

정렬을 하면 행의 순서는 바뀌지만 이름표는 따라 움직인다. 그래서 두 결과가 서로 다른 학생을 가리키게 된다.

Practice 4. Checking the Key

ID가 행을 유일하게 구분하는지 확인한다

```
print(df["ID"].is_unique)
```

```
print(df.duplicated(subset=["ID"]).sum())
```

중복된 행을 확인한 뒤 첫 번째만 남긴다

```
print(df[df.duplicated(subset=["ID"], keep=False)])
```

```
df = df.drop_duplicates(subset=["ID"], keep="first")
```

```
print(df.shape)
```

중복을 지우기 전에 반드시 눈으로 확인한다. 같은 ID가 두 번 나오는 것이 오류일 수도 있지만, 관측 단위가 학생이 아니라는 신호일 수도 있다.

Practice 5. One Column, Two Variables

```
#하이픈을 기준으로 MajorGrade을 두 조각으로 나눈다
parts = df["MajorGrade"].str.split("-", expand=True)
print(parts)

#나눈 결과를 각각 새로운 열로 저장한다
df["Major"] = parts[0]
df["Grade"] = pd.to_numeric(parts[1])

print(df[["ID", "MajorGrade", "Major", "Grade"]])
```

이제 전공별 비교와 학년별 비교를 각각 수행할 수 있다. `expand=True`는 나눈 조각을 여러 열로 펼쳐 준다.

Practice 6. Wide to Long

```
#주차별 점수는 별도의 wide 형태 표에 있다
wide = pd.DataFrame({
    "ID": [201, 202, 203, 204, 205],
    "W1": [70, 65, 88, 60, 80],
    "W2": [74, 73, 90, 66, 84],
    "W3": [79, 77, 91, 68, 86]
})

#주차 열을 하나의 변수와 값으로 펼친다
long = wide.melt(id_vars="ID",
                 var_name="Week",
                 value_name="WeekScore")

print(wide.shape, long.shape)
print(long.head())
```

5행 4열이 15행 3열이 된다. 한 행의 의미가 학생에서 학생-주차로 바뀐 것이다.

Practice 6. Long Back to Wide

```
#melt의 반대 방향 변환
back = long.pivot(index="ID", columns="Week",
                  values="WeekScore")

#index가 된 ID를 다시 일반 열로 되돌린다
back = back.reset_index()

print(back)
```

두 형태는 언제든지 서로 바꿀 수 있다. 보고서에 넣을 때는 wide, 주차별 평균을 계산하거나 그래프를 그릴 때는 long을 사용한다.

Practice 7. Joining Two Tables

```
info = df[["ID", "Major", "Grade", "PriorLevel", "StudyTime"]]  
  
# ID를 기준으로 학생 정보를 주차 기록에 붙인다  
merged = long.merge(info, on="ID", how="left")  
  
# 결합 전후의 행 수를 반드시 비교한다  
print(long.shape, merged.shape)  
  
# 붙지 않은 행이 있는지 확인한다  
print(merged["Major"].isna().sum())  
print(merged.head())
```

행 수가 15로 유지되고 Major에 결측이 없으면 결합이 정상적으로 이루어진 것이다.

Practice 7. When a Join Goes Wrong

```
# 일부 학생만 들어 있는 표를 붙여 본다
partial = pd.DataFrame({
    "ID": [201, 202, 999],
    "Mentor": ["Kim", "Lee", "Park"]
})

# how에 따라 남는 행이 달라진다
left_join = long.merge(partial, on="ID", how="left")
inner_join = long.merge(partial, on="ID", how="inner")

print(left_join.shape, inner_join.shape)
print(left_join["Mentor"].isna().sum())
```

how="left"는 15행을 유지하되 짝이 없는 곳을 결측으로 두고, how="inner"는 짝이 맞는 행만 남긴다. 어떤 학생이 사라졌는지 확인하는 습관이 중요하다.

Practice 8. Summarizing by Group

```
# 전공별로 나누어 한 줄씩 요약한다
summary = merged.groupby("Major").agg(
    n=("ID", "count"),
    mean_score=("WeekScore", "mean"),
    max_score=("WeekScore", "max"),
    mean_study=("StudyTime", "mean")
)

print(summary.round(1))

# 결과의 행 수는 전공의 개수와 같다
print(summary.shape)
```

요약 결과에서 Major는 열이 아니라 행의 이름표가 된다. 일반 열로 되돌리려면 `reset_index()`를 사용한다.

Practice 8. Bringing the Group Value Back

```
# 전공 평균을 각 행에 되돌려 붙인다
merged["MajorMean"] = merged.groupby("Major")["WeekScore"].transform("mean")

# 전공 안에서의 상대적 위치를 계산한다
merged["Gap"] = merged["WeekScore"] - merged["MajorMean"]

print(merged[["ID", "Major", "Week", "WeekScore",
              "MajorMean", "Gap"]].head(8).round(1))
```

요약은 행 수를 그룹 수로 줄이고, 되돌리기는 행 수를 그대로 유지한다. 두 결과의 shape을 비교해 보면 차이가 분명해진다.

Putting It Together

```
# 정리된 데이터로 주차별, 전공별 평균을 표로 만든다
table = merged.pivot_table(index="Major",
                             columns="Week",
                             values="WeekScore",
                             aggfunc="mean")

print(table.round(1))

# 학습 시간이 긴 학생의 점수가 높은지 확인한다
print(merged["StudyTime"].corr(merged["WeekScore"]).round(3))
```

1주차에서 배운 상관계수 계산이, 정리된 데이터 위에서 비로소 의미를 가진다. 문자로 저장된 학습 시간과 -99가 남아 있었다면 이 계산은 불가능했거나 잘못된 값을 주었을 것이다.

From Raw Table to Analysis

2-2에서 수행한 작업을 순서대로 정리하면 다음과 같다.

단계	작업	확인한 내용
1	타입과 척도 확인	어떤 요약이 가능한가
2	수치·날짜 변환	계산이 가능한 형태인가
3	결측 정리	몇 개이며 어떻게 처리했는가
4	중복 확인	한 행이 무엇인가
5	열 분리	변수가 하나씩 들어 있는가
6	형태 변환	지금 작업에 맞는 배치인가
7	표 결합	행 수가 유지되었는가
8	그룹 비교	요약인가 되돌리기인가

실제 분석에서 이 과정이 전체 작업 시간의 대부분을 차지한다.

Checklist for Week 2

- 네 가지 척도를 구분하고, 각 척도에 맞는 요약 방법을 고를 수 있는가?
- 저장 형식과 실제 의미가 어긋난 열을 찾아낼 수 있는가?
- 문자로 된 숫자와 날짜를 적절한 타입으로 바꿀 수 있는가?
- 결측을 확인하고, 선택한 처리 방법의 이유를 설명할 수 있는가?
- 위치로 행을 고르는 것과 이름표로 고르는 것의 차이를 아는가?
- key의 유일성을 확인하고 중복을 처리할 수 있는가?
- Tidy data 원칙에 따라 한 열에 섞인 변수를 분리할 수 있는가?
- Wide와 Long을 서로 변환하고, 각각이 편리한 상황을 말할 수 있는가?
- 표를 결합한 뒤 행 수와 결측을 확인하는가?
- 그룹 요약과 되돌리기의 차이를 설명할 수 있는가?

Week 2 Summary

- 변수의 척도가 사용할 수 있는 요약 방법을 결정한다.
- 저장 형식과 의미가 다를 수 있으므로 타입을 먼저 확인하고 변환한다.
- 결측은 여러 모습으로 나타나며, 처리 방법과 근거를 함께 기록한다.
- 한 행의 의미와 key의 유일성이 표 구조의 기준이다.
- Tidy data 원칙과 형태 변환으로 표를 분석 가능한 상태로 만든다.
- 결합과 그룹 비교의 결과는 항상 행 수를 확인하며 점검한다.

다음 주에는 이렇게 정리한 데이터를 바탕으로 기술통계와 분포의 특성을 다룬다.

Review Exercise

관심 있는 주제로 작은 데이터셋을 직접 만들고, Topic 1-8을 순서대로 적용하시오.

1. 최소 6명 이상의 정보를 담은 표를 만들되, 문자로 저장된 숫자 열과 날짜 열을 각각 하나씩 포함하시오.
2. 결측을 뜻하는 값과 중복된 행을 의도적으로 하나씩 넣으시오.
3. 각 열의 척도를 표로 정리하고, 순서가 있는 범주에는 순서를 지정하시오.
4. 숫자 열과 날짜 열을 적절한 타입으로 변환하시오.
5. 결측을 확인하고 처리한 뒤, 선택한 방법의 이유를 서술하시오.
6. 중복을 확인하고 정리한 뒤, 행 수의 변화를 기록하시오.
7. 반복 측정 값을 담은 **wide** 표를 따로 만들어 **long**으로 변환하고, 첫 번째 표와 결합하시오.
8. 범주별 요약표를 만들고, 각 관측이 소속 그룹 평균과 얼마나 차이 나는지 계산하시오.

Review Exercise: 답안 예시 1

1-2. 데이터 만들기

```
import numpy as np
import pandas as pd

# 관측 단위는 회원 한 명, key는 ID
member = pd.DataFrame({
    "ID": [1, 2, 3, 4, 5, 6, 3],
    "Center": ["East", "East", "West", "West",
               "South", "South", "West"],
    "Plan": ["basic", "premium", "basic", "premium",
             "basic", "premium", "basic"],
    "Minutes": ["50", "80", "unknown", "-99",
                "65", "95", "unknown"],
    "JoinDate": ["2026-01-05", "2026-01-05", "2026-02-02",
                 "2026-02-02", "2026-03-02", "2026-03-02",
                 "2026-02-02"]
})
```

Review Exercise: 답안 예시 2

3. 각 열의 척도

Variable	의미	척도	비고
ID	회원 식별번호	identifier	key
Center	소속 지점	nominal	순서 없음
Plan	이용권 종류	ordinal	basic < premium
Minutes	주당 이용 시간(분)	ratio	문자로 저장됨
JoinDate	가입일	interval	문자로 저장됨

Plan은 두 단계뿐이지만 basic보다 premium이 상위 등급이라는 순서가 있으므로 ordinal로 본다. 순서를 명시해 두면 정렬과 비교가 우리가 정한 기준대로 이루어진다.

Review Exercise: 답안 예시 3

3-4. 척도 지정과 타입 변환

순서가 있는 범주임을 명시한다

```
member["Plan"] = pd.Categorical(member["Plan"],  
                                categories=["basic", "premium"],  
                                ordered=True)
```

문자로 저장된 숫자를 수치형으로 바꾼다

```
member["Minutes"] = pd.to_numeric(member["Minutes"],  
                                   errors="coerce")
```

문자로 저장된 날짜를 날짜형으로 바꾼다

```
member["JoinDate"] = pd.to_datetime(member["JoinDate"])
```

```
print(member.dtypes)
```

Review Exercise: 답안 예시 4

5-6. 결측과 중복 처리

```
# 약속된 숫자를 결측으로 바꾸고 개수를 확인한다
member["Minutes"] = member["Minutes"].replace(-99, np.nan)
print(member["Minutes"].isna().sum())

# 대체 여부를 남긴 뒤 지점별 중앙값으로 채운다
member["Imputed"] = member["Minutes"].isna()
member["Minutes"] = member["Minutes"].fillna(
    member.groupby("Center")["Minutes"].transform("median")
)

# 중복을 확인한 뒤 첫 행만 남긴다
print(member.duplicated(subset=["ID"]).sum())
member = member.drop_duplicates(subset=["ID"], keep="first")
print(member.shape)
```

Review Exercise: 답안 예시 5

5. 결측 처리 방법을 선택한 이유

- “unknown”과 -99는 모두 이용 시간이 0분이라는 뜻이 아니라 기록이 없다는 뜻이므로, 0으로 채우지 않았다.
- 회원 수가 적어 해당 행을 제거하면 지점별 비교에 필요한 표본이 부족해지므로 대체 방법을 선택하였다.
- 전체 평균 대신 지점별 중앙값을 사용한 이유는 지점마다 이용 시간의 수준이 다를 수 있고, 중앙값이 극단값의 영향을 덜 받기 때문이다.
- 대체 여부를 Imputed 열로 남겨, 이후 분석에서 대체한 값을 제외한 결과와 비교할 수 있도록 하였다.

중요한 것은 특정 방법을 고르는 일이 아니라, 선택의 이유와 그 영향을 함께 기록하는 일이다.

Review Exercise: 답안 예시 6

7. 형태 변환과 결합

```
# 월별 방문 횟수를 담은 wide 형태의 표
visit = pd.DataFrame({
    "ID": [1, 2, 3, 4, 5, 6],
    "M1": [8, 12, 5, 14, 9, 15],
    "M2": [10, 11, 6, 13, 8, 16],
    "M3": [9, 13, 7, 15, 10, 14]
})

# 월 열을 행으로 펼친다
long = visit.melt(id_vars="ID", var_name="Month",
                  value_name="Visits")
print(visit.shape, long.shape)

# 회원 정보를 붙이고 행 수를 확인한다
panel = long.merge(member, on="ID", how="left")
print(panel.shape, panel["Center"].isna().sum())
```

Review Exercise: 답안 예시 7

8. 그룹 요약과 상대적 위치

지점별 요약: 행 수는 지점의 개수와 같다

```
summary = panel.groupby("Center").agg(  
    n=("ID", "count"),  
    mean_visits=("Visits", "mean"),  
    mean_minutes=("Minutes", "mean")  
)
```

```
print(summary.round(1))
```

지점 평균을 각 행에 되돌려 붙인다

```
panel["CenterMean"] = panel.groupby("Center")["Visits"].transform("mean")  
panel["Gap"] = panel["Visits"] - panel["CenterMean"]
```

지점 평균보다 방문이 적은 관측만 확인한다

```
print(panel[panel["Gap"] < 0][["ID", "Center", "Month",  
                               "Visits", "Gap"]].round(1))
```

Review Exercise: 결과 해석 예시

- **척도**: Plan에 순서를 지정하여 basic과 premium의 상하 관계가 정렬과 비교에 반영되도록 하였다.
- **타입 변환**: Minutes를 수치형으로 바꾸는 과정에서 결측 2건이 확인되었으며, 이는 원자료에 unknown과 -99로 기록되어 있던 값이다.
- **결측 처리**: 지점별 중앙값으로 대체하고 대체 여부를 별도 열로 남겨, 대체가 결과에 미친 영향을 사후에 확인할 수 있게 하였다.
- **중복 처리**: ID 3이 두 번 기록되어 있어 7행이 6행으로 줄었다. 중복을 남겨 두었다면 해당 회원이 평균에 두 번 반영되었을 것이다.
- **형태 변환과 결합**: wide 6행이 long 18행이 되었고, 회원 정보를 붙인 뒤에도 18행이 유지되고 결측이 없으므로 key가 정상적으로 대응되었다.
- **그룹 비교**: 지점별 평균 방문 횟수를 비교하였고, Gap 열을 통해 각 관측이 소속 지점 평균보다 높은지 낮은지 판단할 수 있다.

본 교육콘텐츠는 교육부와 한국연구재단의 지원을 받아 제작한 AI융합
인재양성사업단의 콘텐츠입니다.